SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing – Department of Computer Science & Software Engineering

SE3082 – Parallel Computing | Lab Report 10

Student Name / ID:	IT24610803	**Academic Year / Sem:**	Year 3, Semester 2
Module:	SE3082 – Parallel Computing	**Lab Topic:**	CUDA Timing, CHECK Macro, 2D & 3D Grids
Environment:	Google Colab – NVIDIA Tesla T4 GPU (~15 GB VRAM, Compute Capability 7.5)		

Exercise 1: Timing a GPU Program

In this exercise, I measured the execution time of CUDA kernels using a CPU wall-clock timer (`gettimeofday`). Since GPU kernel launches are asynchronous, the CPU program will continue immediately without waiting for the GPU to finish. Therefore, calling `cudaDeviceSynchronize()` right after the kernel launch is necessary so that the CPU waits for all GPU threads to complete before taking the final time stamp.

Part (a) Output of `ex1_timed.cu`:

```
83 + 77 = 160
86 + 34 = 120
77 + 90 = 167
15 + 26 = 41
93 + 24 = 117
Verification: 0 mismatches out of 512
Time taken - 0.0000421
```

Experiment – Removing `cudaDeviceSynchronize()`:

When I deleted the `cudaDeviceSynchronize();` line and ran the program again, the printed **Time taken dropped to 0.0000040 seconds** (almost instantaneous). This is because the clock stopped immediately after the launch command was sent to the GPU, without timing the actual computation. However, the result still verified with 0 mismatches because `cudaMemcpy()` from Device to Host automatically synchronizes the device before reading data.

Part (b) – Execution Times of Lab 9 Programs:

Program	Configuration	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)
Exercise 3	512 blocks × 1 thread	0.0000482	0.0000475	0.0000479	0.0000479
Exercise 4	1 block × 512 threads	0.0000310	0.0000305	0.0000308	0.0000308
Exercise 5	10M elements (19,532 × 512)	0.0016250	0.0016180	0.0016210	0.0016213
Exercise 6	100M elements (313 × 625 × 512)	0.0158400	0.0157900	0.0158100	0.0158133

Predict Before Measure Answers:
1. Ex 3 vs Ex 4: Exercise 4 (1 block of 512 threads) is faster than Exercise 3 (512 blocks of 1 thread) because

Exercise 4 utilizes 16 full 32-thread warps on a single SM with 100% warp efficiency. In contrast, Exercise 3 launches 512 blocks where each warp has only 1 active thread and 31 idle lanes (only 3.125% warp efficiency), leading to extra block scheduling overhead.
2. Ex 6 vs Ex 5: Exercise 6 has 10× more elements and takes roughly 9.8× longer (~0.0158s vs ~0.0016s), which shows near-linear scaling once all GPU cores are saturated.

Part (c) – Kernel Time vs Data Transfer (100 Million Elements):

```
Data Size: 381 MB per array (3 arrays = 1144 MB total)
Kernel-only time:    0.0142100 seconds
Whole-sequence time: 0.2645000 seconds (Copy-in + Compute + Copy-out)
PCIe Transfer Time:  0.2502900 seconds (94.6% of total time)
```

Think Question Answer:
The whole sequence time is approximately **18 times longer** than the kernel execution time alone. This proves that transferring data back and forth across the PCIe bus is the main performance bottleneck. Therefore, GPUs are best suited for tasks that have high arithmetic intensity (many calculations per element) rather than simple single-pass operations.

Exercise 2: Catching Errors with the CHECK() Macro

To prevent silent errors in CUDA, I wrapped all CUDA runtime functions with the CHECK() macro. If any CUDA call fails, the macro prints the file name, line number, error code, and error description, and safely terminates the program.

```
// Macro definition used:
#define CHECK(call) do {     const cudaError_t error = (call);     if (error != cudaSuccess) {
printf("Error: %s:%d, ", __FILE__, __LINE__);         printf("code:%d, reason: %s\n", error,
cudaGetErrorString(error));         exit(1);     } } while (0)
```

Deliberate Error Experiments (Break it on Purpose):

Experiment	Code Change	Observed Output from CHECK()
Experiment A: Too many threads	`add<<<N, 2048>>>`	`Error: ex2_check.cu:46, code:9, reason: invalid configuration argument`
Experiment B: Impossible allocation	`cudaMalloc(..., (size_t)1 << 50)`	`Error: ex2_check.cu:31, code:2, reason: out of memory`
Experiment C: Without CHECK macro	Omitted CHECK on Exp A	No error message printed; kernel silently fails and prints 512 mismatches.

Why use `do { ... } while(0)`?
Using `do { ... } while(0)` allows the macro to act as a single C statement with a semicolon at the end. This prevents syntax errors when used inside conditional statements like `if (...) CHECK(...); else ...`.

Exercise 3: Scaling a Picture with a 2D Grid (`PictureKernel`)

In this exercise, I processed an $800 imes 600$ picture (480,000 pixels) where each pixel is stored as an `unsigned char` (0 to 255). The kernel multiplies each pixel by 2.0 and saturates the result at 255 if it exceeds 255.

Grid Configuration:

- **Threads per Block:** `dim3 threadsPerBlock(16, 16);` (256 threads/block)
- **Grid Size:** $x = (800 + 15) / 16 = \mathbf{50 ext{ blocks}}$, $y = (600 + 15) / 16 = \mathbf{38 ext{ blocks}}$
- **Total Blocks:** $50 imes 38 = 1,900$ blocks (486,400 threads). The boundary check `if (Row < height && Col < width)` successfully stops the extra 6,400 threads from going out of bounds.

Program Output:

```
Grid: 50 x 38 blocks of 16 x 16 threads
Original 50x50 portion (first 10 values of row 0):
103 198 105 115  81 255  74 236  41 205 ...

Updated 50x50 portion (first 10 values of row 0):
206 255 210 230 162 255 148 255  82 255 ...
```

```
Verification: 0 mismatches out of 480000
Pixels capped at 255: 240121
Time taken - 0.0001840
```

Predict Before You Run:
Since random values are between 0 and 255, any original pixel value ≥ 128 will double to ≥ 256 and get capped at 255. Because random numbers are uniformly distributed, roughly half of the pixels (~240,000) are capped at 255 (actual run was 240,121).

Exercise 4: Multiplying Two 3D Matrices Element-by-Element

I extended the 2D matrix exercise into a 3D volume of size $500 imes 500 imes 500$ ($125,000,000$ integers, 500 MB per matrix). Each thread calculates its 3D coordinates (x, y, z) and maps them to a flat row-major index:

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
int z = blockIdx.z * blockDim.z + threadIdx.z;

if (x < nx && y < ny && z < nz) {
    size_t idx = ((size_t)z * ny + y) * nx + x;
    c[idx] = a[idx] * b[idx];
}
```

Grid Configuration:

- **Threads per Block:** `dim3 threadsPerBlock(8, 8, 8);` (512 threads/block)
- **Grid Size:** $(500 + 7) / 8 = \mathbf{63 ext{ blocks}}$ in $x, y, z \implies 63 imes 63 imes 63 = \mathbf{250,047 ext{ blocks}}$
- **Total Threads:** $250,047 imes 512 = 128,024,064$ threads (3,024,064 extra threads protected by the triple boundary guard).

Program Output:

```
Grid: 63 x 63 x 63 blocks
C[499][498][0] = 76 x 15 = 1140
C[499][498][1] = 86 x 41 = 3526
...
C[499][499][498] = 31 x 73 = 2263
C[499][499][499] = 47 x 68 = 3196
Verification: 0 mismatches out of 125000000
Time taken - 0.0241800
```

Think Question Answer:
Comparing Lab 9 Exercise 6 (2D, 100M elements) taking ~0.0158s with this 3D exercise (125M elements) taking ~0.0241s, the computation time scales proportionately with the element count ($1.25 imes$). The 3D thread arrangement behaves similarly to 2D because threads in the same warp access consecutive x-coordinates, leading to fully coalesced memory transactions along the fastest-changing dimension.

Conclusion & Verification Summary

Exercise	Task / Concept	Result / Mismatches	Status
Exercise 1	CPU Timer & cudaDeviceSynchronize()	0 mismatches out of 512	PASSED
Exercise 2	CHECK() Macro Error Handling	0 mismatches out of 512	PASSED
Exercise 3	2D PictureKernel ($800 imes 600$)	0 mismatches out of 480,000	PASSED
Exercise 4	3D Volume Kernel ($500 imes 500 imes 500$)	0 mismatches out of 125,000,000	PASSED